

TRABALHO DE CONCLUSÃO DE CURSO
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

ANÁLISE COMPARATIVA DE ARDUINOS NA IMPLEMENTAÇÃO DE SISTEMAS EMBARCADOS PARA MONITORAMENTO DE TREMORES NA DOENÇA DE PARKINSON

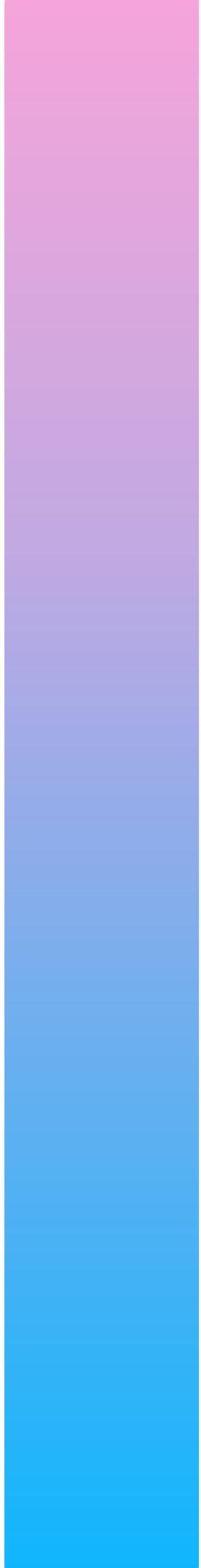
Manuele Sabatini Christófalo

Orientador: Prof. Dr. Clayton Reginaldo Pereira



SUMÁRIO



1. Introdução
 2. Problemática
 3. Justificativa
 4. Fundamentação Teórica
 5. Metodologia
 6. Resultados Experimentais
 7. Considerações Finais
- 

INTRODUÇÃO



A **Doença de Parkinson (DP)**, condição neuromotora mais prevalente em todo o mundo (FEIGIN et al., 2019), é caracterizada pela degeneração dos neurônios dopaminérgicos. Isso implica em seus sintomas motores característicos (KLEIN; BATTAGELLO; CARDOSO, 2019).

O **monitoramento dos sintomas** é imprescindível para a manutenção da saúde dos indivíduos diagnosticados. Por se tratar de uma doença neurológica, **impactada por múltiplos fatores** como idade, prática de atividades físicas e condições pré-existentes, a gravidade e evolução dos sintomas são completamente individuais, de forma que as suas determinações acabam representando um **desafio aos médicos** (ARMSTRONG; OKUN, 2020).

SOBRE O TRATAMENTO

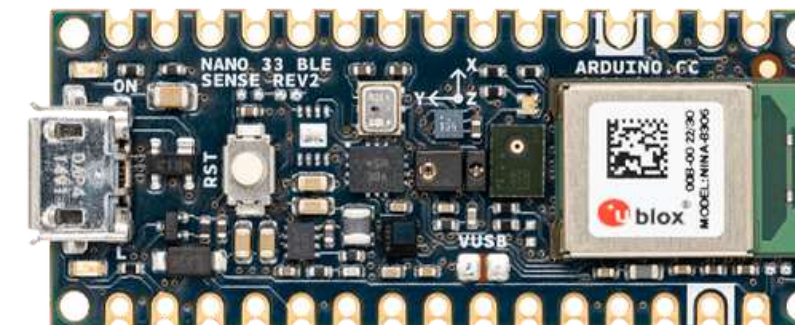
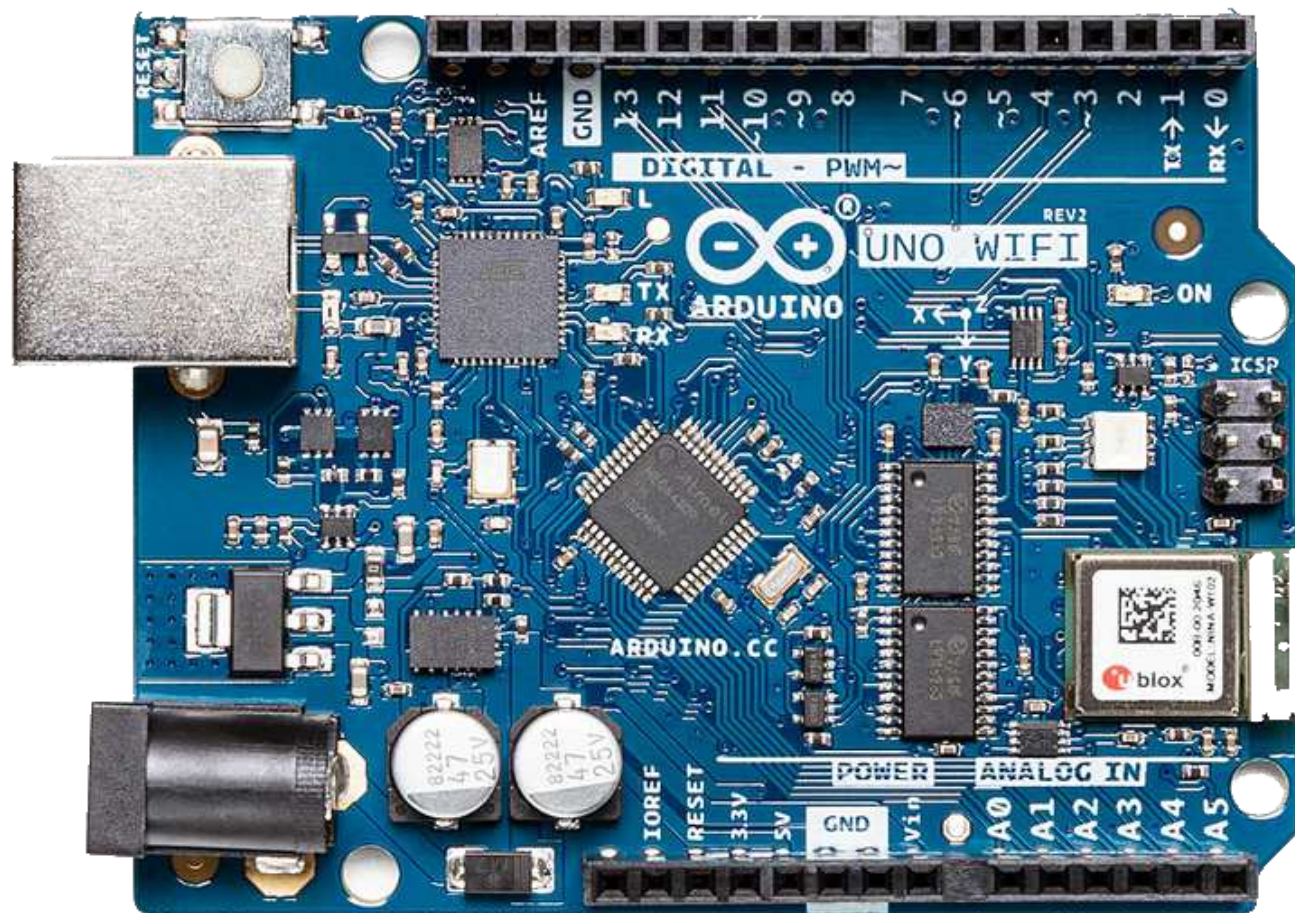
Capturar os níveis de tremor e realizar suas análises pode contribuir significativamente para a compreensão da fisiopatogênese e do funcionamento cerebral do paciente (SUN et al., 2023).

Destaca-se o uso emergente de tecnologias assistivas para diagnóstico e acompanhamento de doenças e outras condições médicas, em especial, os ***wearables***.

Especificamente para a DP, como constatado por Rovini et. al (ROVINI; MAREMMANI; CAVALLO, 2017), **sensores como giroscópios e acelerômetros em *wearables* possuem grande potencial no contexto da doença**, já que permitem a captura de movimentações e suas intensidades, o que condiz com os métodos requeridos pelas principais escalas avaliativas.

SOBRE A PROPOSTA

Comparar dois arduinos em termos de eficiência tecnológica, economia, custo e espaço em um sistema embarcado *wearable* voltado para o acompanhamento da DP (Arduino UNO R3 x Nano 33 BLE Sense Rev2).



Fonte: Arduino Official Store

PROBLEMÁTICA



ESTADO DA ARTE

Apesar do número considerável de pesquisas em técnicas computacionais para DP, constata-se uma **exiguidade ao tratar da análise de sinais**.

ACOMPANHAMENTO DA DOENÇA

Urge um acompanhamento sólido, capaz de **quantificar objetivamente o nível da doença** e resposta do paciente ao tratamento. A questão é definir **qual o melhor método** para construir um dispositivo especializado em promover esse acompanhamento, visando precisão, com baixo consumo energético e não invasivo àqueles que passam pela terapia.

JUSTIFICATIVAS



ENRIQUECIMENTO MÉDICO

O avanço na tecnologia de sensores e sistemas embarcados possibilita um **monitoramento contínuo** dos tremores da doença, fornecendo dados detalhados que auxiliam na **adaptação do tratamento** às necessidades individuais de cada paciente.

ESTUDO TÉCNICO

A revisão elaborada não apenas fortalece a aplicação da eletrônica e da computação na área da saúde, mas também **fornece diretrizes importantes** para a escolha de plataformas adequadas em projetos futuros de monitoramento biomédico. Além disso, a pesquisa apresenta soluções acessíveis e viáveis para o monitoramento dos sintomas.

FUNDAMENTAÇÃO



WEARABLES APLICADOS NA SAÚDE

- Smartwatches, pulseiras de fitness e dispositivos de monitoramento de saúde
- Técnicas de classificação de padrões para rastreamento do sono na busca de avaliar os dispositivos atuais aplicados a essa finalidade (ZAMBOTTI et al., 2023)
- CueBand (WALKER et al., 2024), Kinesia 360 (Great Lakes NeuroTechnologies, 2024), Cala Trio (Cala Health, 2024)

ARDUINO

Proposta de atuar como uma ferramenta de baixo custo e de fácil uso para estudantes, artistas, e pessoas sem ou com baixa instrução em programação e hardware (Arduino, 2021).

Hoje, a Arduino já possui diversos modelos de placas e uma comunidade global de usuários. Sua característica open-source permite um crescimento rápido, acessível e amplo das possibilidades através da ferramenta.

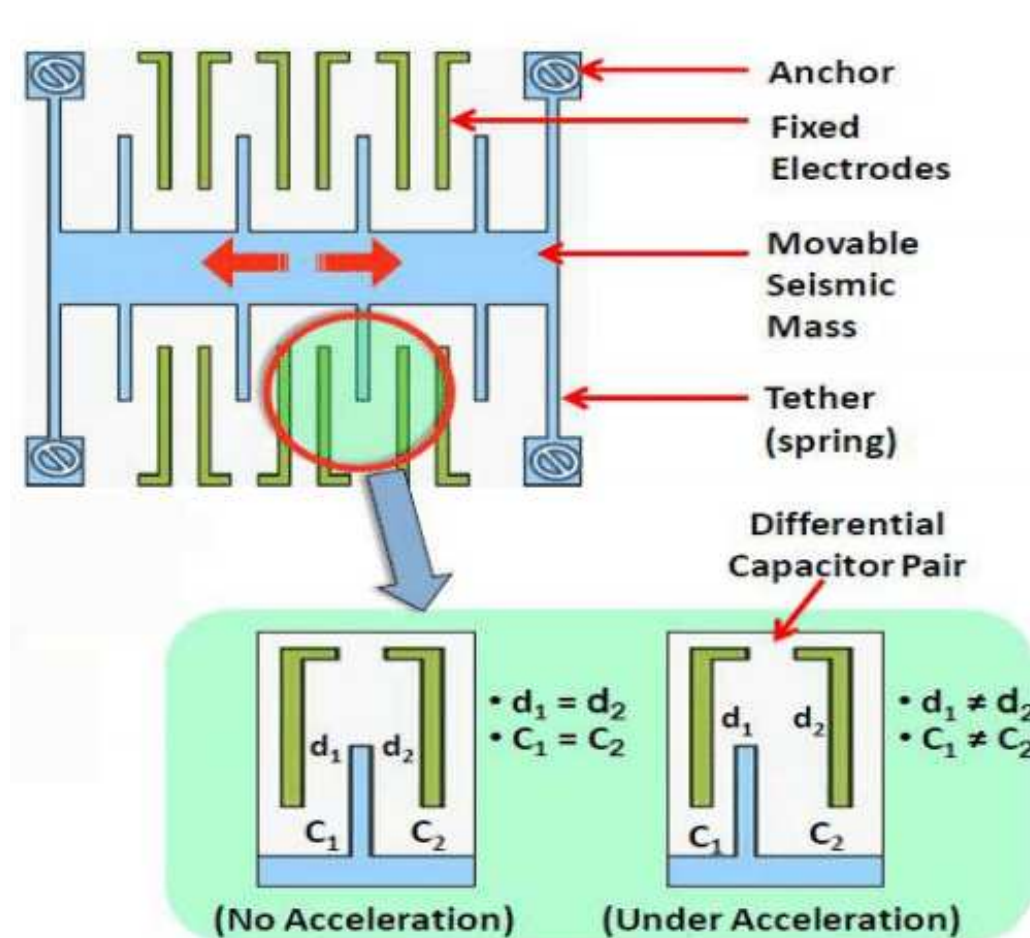


Fonte: Arduino Blog

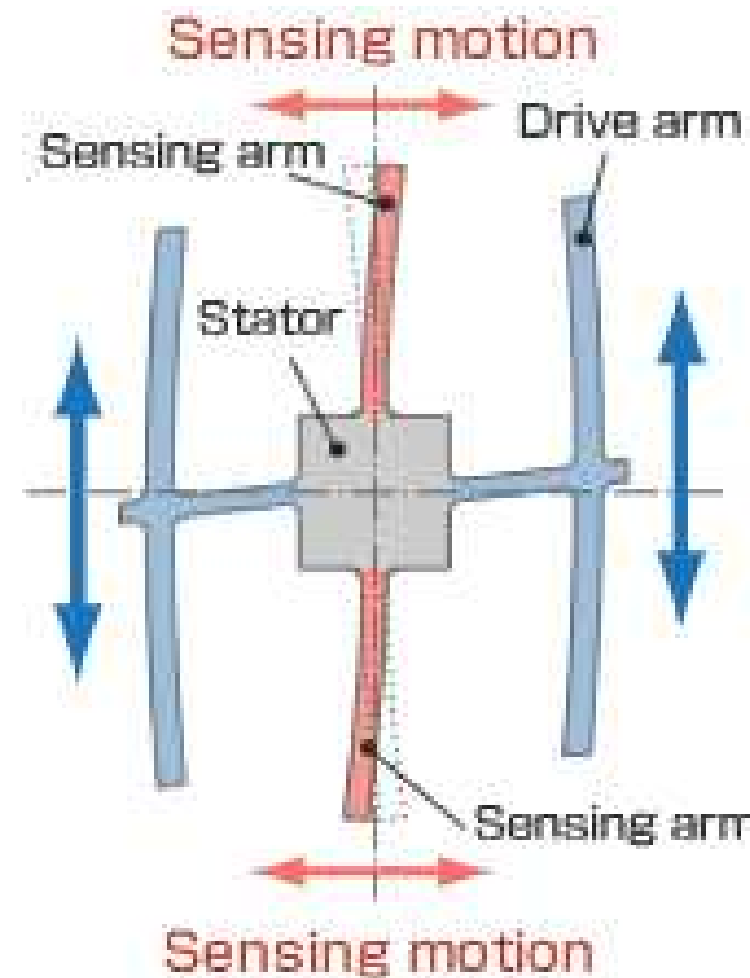
"One board to rule them all: History of the Arduino UNO"

FUNIONAMENTO DOS SENSORES

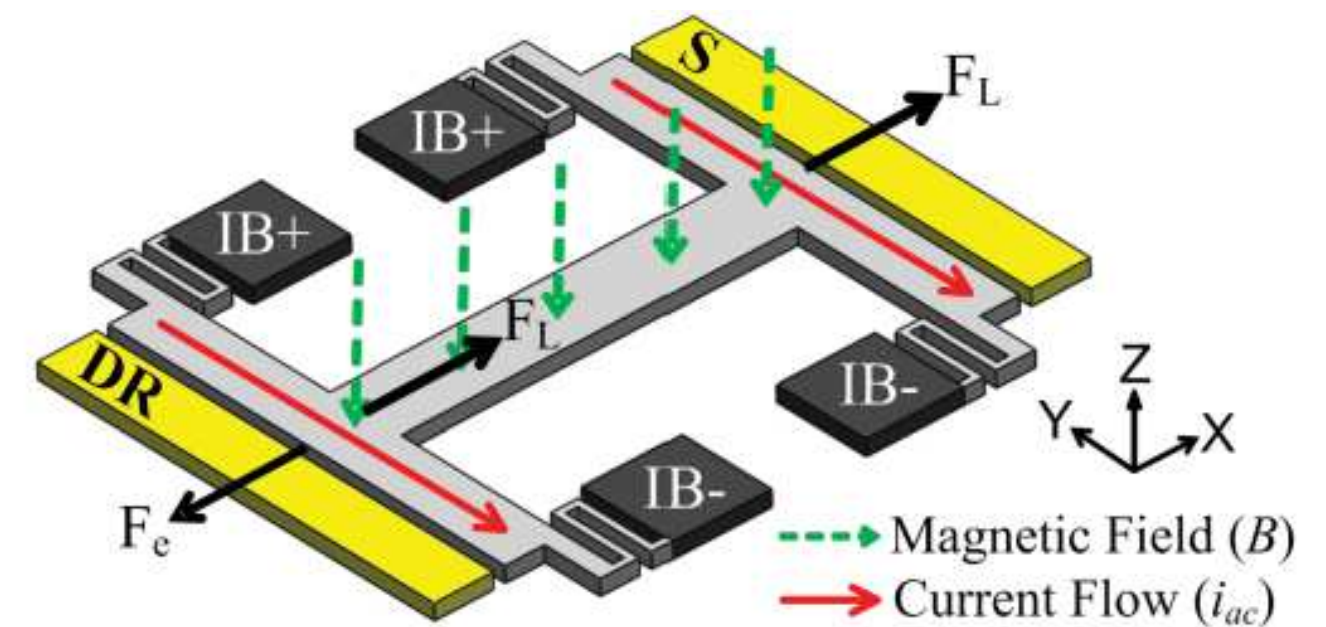
- **Arduino Nano 33 BLE Sense Rev 2:** *Inertial Measurement Unit (IMU)* - Giroscópio + Acelerômetro + Magnetômetro
- **Arduino UNO:** *Motion Processing Unit (MPU)* - Giroscópio + Acelerômetro



Fonte: AMERINI et al., 2016



Fonte: EPSON, 2024



Fonte: SONER SONMEZOGLU et al., 2017

PROCESSAMENTO E OBTENÇÃO DOS DADOS

Número digital adimensional unidade métrica significativa.

Este processo é fundamentado em dois parâmetros-chave: a sensibilidade (ou fator de escala) do sensor e o viés (ou *bias*).

- **Acelerômetro:**

$$A_{m/s^2} = A_g \times 9.81$$

- **Giroscópio:**

$$\theta_k = \theta_{k-1} + (\omega_k \times \Delta t)$$

- **Magnetômetro:**

$$V_H = \frac{I \cdot B_{\perp}}{n \cdot q \cdot d}$$

Vale ressaltar que, diferente do acelerômetro, o giroscópio mede uma **taxa de variação** (integração por tempo). Este cálculo é fundamental, mas também é a principal fonte de erro do giroscópio. **Pequenos erros no viés são acumulados a cada passo da integração**, resultando em um fenômeno conhecido como *drift* (deriva), onde a orientação calculada diverge progressivamente da orientação real ao longo do tempo.

FILTRO DE KALMAN

Algoritmo estatístico que estima o estado de um sistema dinâmico a partir de medições ruidosas e incompletas. Permite a combinação de informações (fusão sensorial ponderada), estimações em tempo real e manejo de ruído.

- **Acelerômetro:** Estima o ângulo atual com base na gravidade, mas sofre interferência de outros sinais (bom a longo prazo, ruim para curto);
- **Giroscópio:** Mede velocidade angular através do tempo, sofrendo acúmulo de viés (bom para curto prazo, ruim para longo prazo).

→ Taxa angular do Giroscópio prevê o angulo atual | **Predição**

→ Correção da estimativa utilizando o Acelerômetro/Magnetômetro | **Correção**

→ Ponderação com base nas incertezas | **Definição de Q e R**

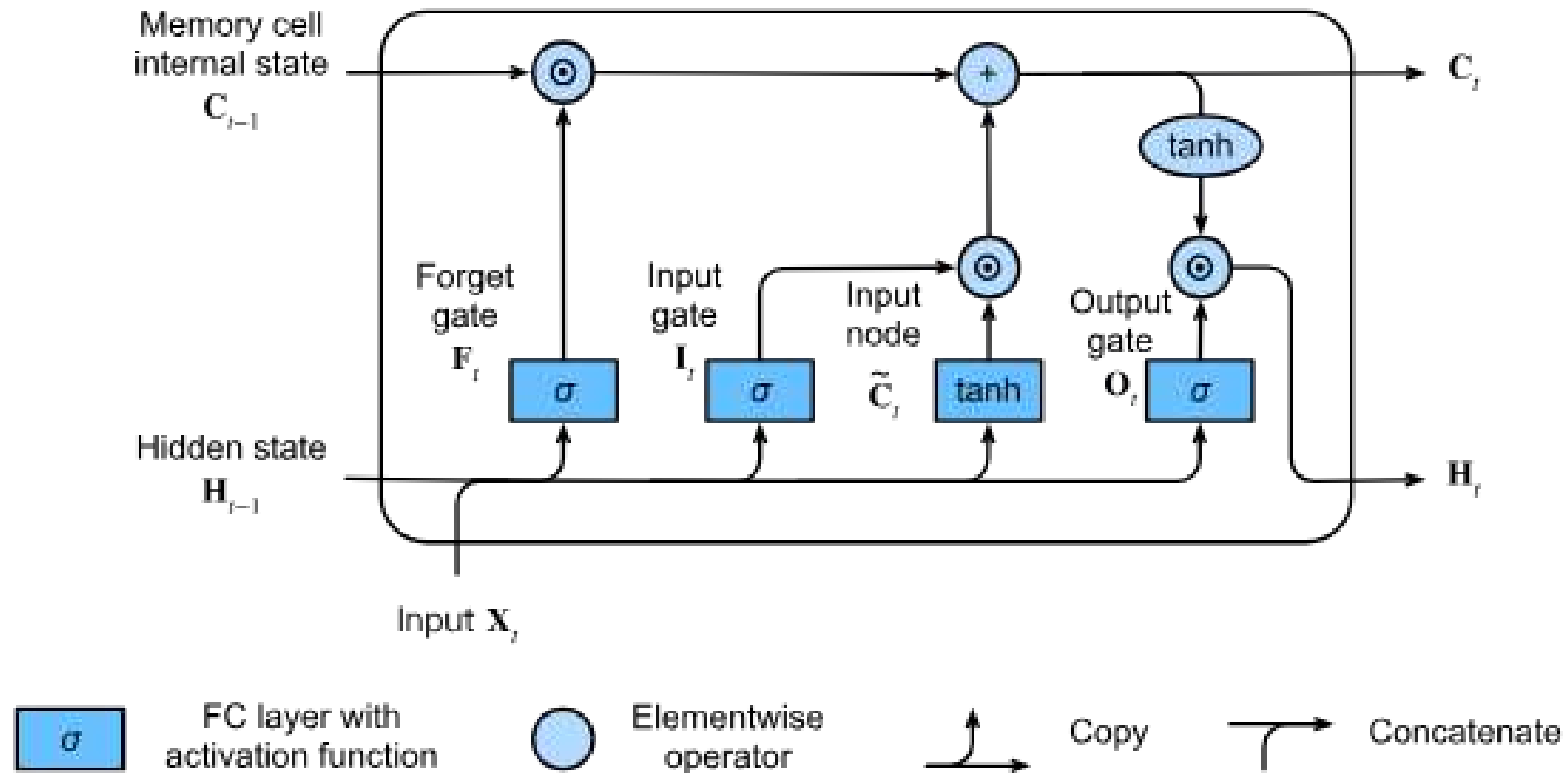
MACHINE LEARNING E REDES NEURAIS RECORRENTES (RNNs)

O Machine Learning (ML) é dedicado ao desenvolvimento de algoritmos e modelos computacionais que permitem a um sistema aprender e aprimorar seu desempenho a partir da análise de dados, sem a necessidade de ser explicitamente programado para cada tarefa (MITCHELL, 1997).

As Redes Neurais Recorrentes (Recurrent Neural Networks - RNNs) representam uma classe de redes neurais artificiais **projetada especificamente para lidar com dados de séries temporais**. As RNNs possuem *loops* que lhes permitem manter um estado interno, ou **memória**, das informações processadas anteriormente (GOODFELLOW; BENGIO; COURVILLE, 2016).

Para superar essa limitação, foram desenvolvidas arquiteturas mais sofisticadas, como as redes Long Short-Term Memory (LSTM) (HOCHREITER; SCHMIDHUBER, 1997).

LONG SHORT-TERM MEMORY (LSTM)



Fonte: LIN; YEAP; KIRINGA, 2022

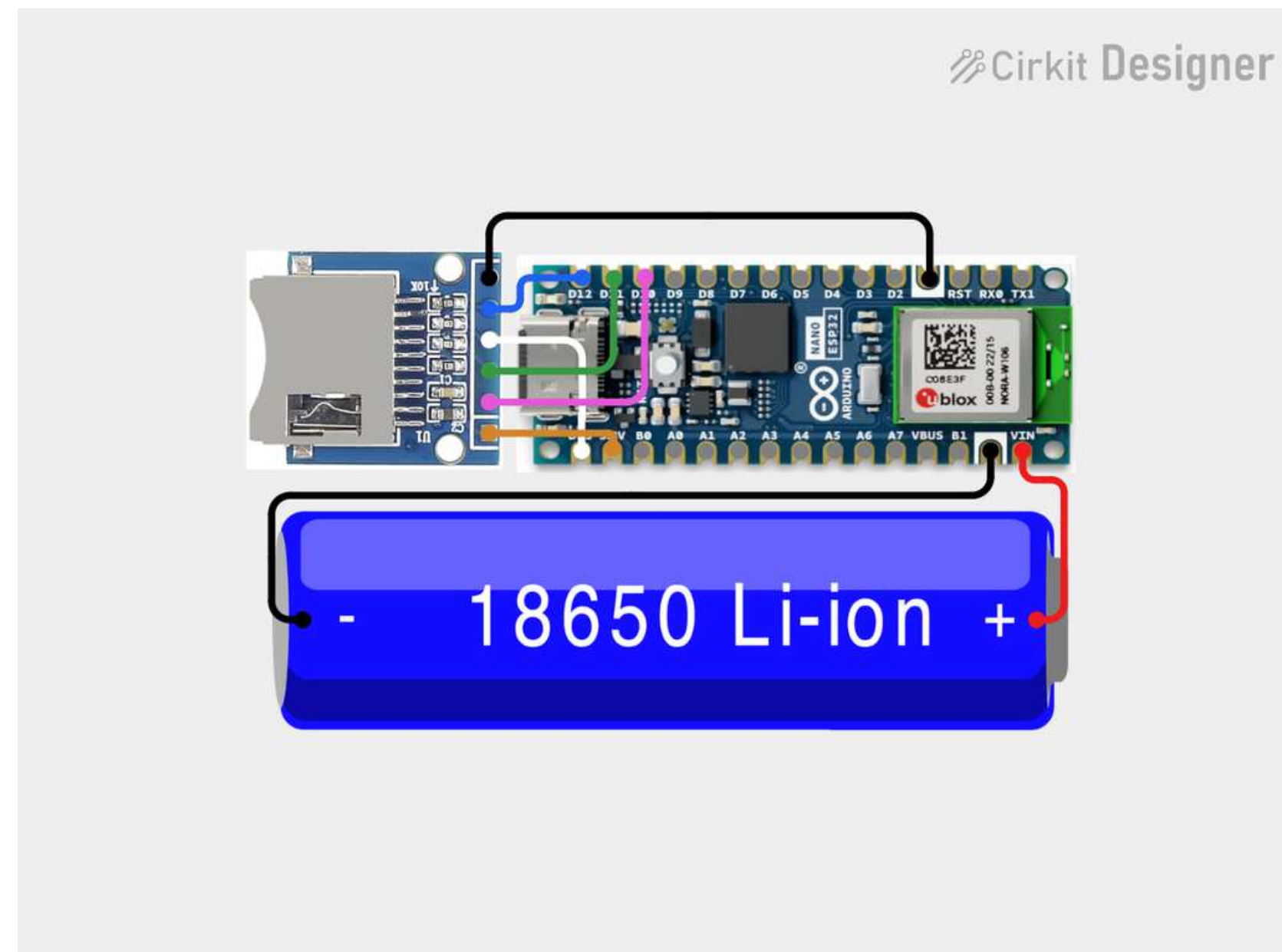
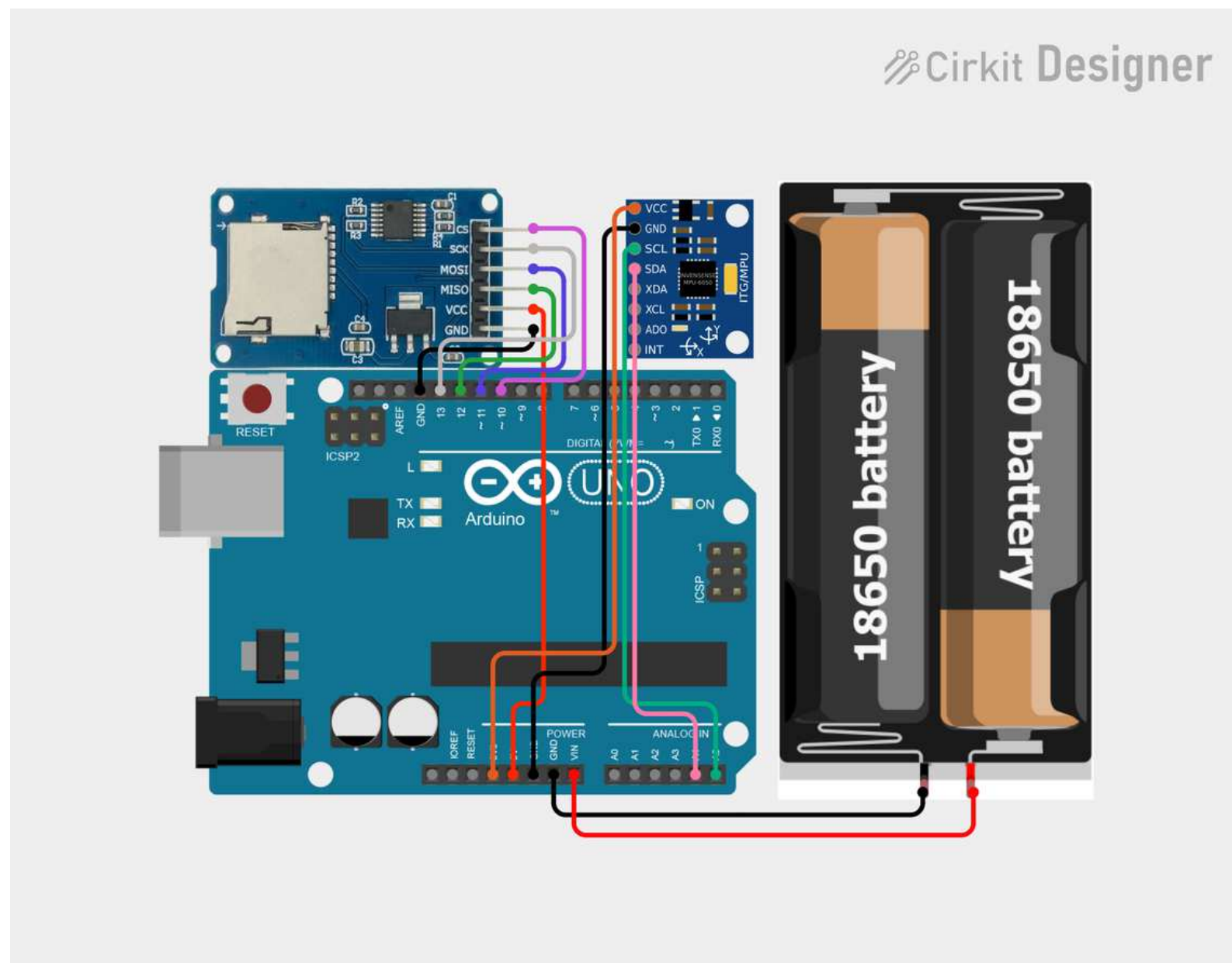
METODOLOGIA



VALIDAÇÃO DOS SENSORES E COMPONENTES

- **Protótipos iniciais:** Arduinos + Baterias CR2032 + Módulos + Filtro de Tensão
- Testes iniciais com o Arduino alimentado pelo computador
- Os primeiros testes verificavam o funcionamento isolado dos módulos. Os subsequentes consideravam o sistema inteiro - a início, conectado ao computador, depois, de modo completamente independente.

PROTÓTIPOS FINAIS



Fonte: elaborada pela autora

FUNIONAMENTO E FIRMWARE

Desenvolvimento dos firmwares finais, responsáveis por orquestrar a aquisição, o processamento e o armazenamento das movimentações do antebraço dos usuários.

- **Arquivo utilizado:** novo arquivo a cada *restart* para evitar sobrescrita;
- **Leitura dos dados:** realizada pela IMU ou MPU;
- **Filtro de Kalman:** utilizado para a fusão ponderada dos sensores;
- **Lógica de armazenamento:** armazenamento em blocos de 10 leituras (~1s) através de um *buffer*.

DECLARAÇÃO DAS VARIÁVEIS

```
// 1. DECLARACAO DAS VARIAVEIS BASE -----
#include "Arduino_BMI270_BMM150.h"
#include <SPI.h>
#include <SD.h>

float AccX, AccY, AccZ;           // Valores lidos pelo acelerometro
float GyroX, GyroY, GyroZ;       // Valores lidos pelo giroscopio
float MagX, MagY, MagZ;          // Valores lidos pelo magnetômetro

int base = 0;                    // Zero do grafico

float tempoDelta, tempoInicial, tempoAtual, tempoAnterior, tempoSegundos; // Variaveis de tempo para calculo do angulo (vel = a°/s)
float AnguloAccX, AnguloAccY;    // Angulos calculados para o acelerometro
float AnguloMagZ;               // Angulo do campo magnetico terrestre (Yaw)
float rotX, rotY, rotZ;         // Angulo de rotacao total do braco (roll, pitch, yaw)

float kalmanEstadoX = 0, kalmanIncertezaX = 1; // Variaveis do filtro de Kalman para X (roll)
float kalmanEstadoY = 0, kalmanIncertezaY = 1; // Variaveis do filtro de Kalman para Y (pitch)
float kalmanEstadoZ = 0, kalmanIncertezaZ = 1; // Variaveis do filtro de Kalman para Z (yaw)

float varGyro = 0.0016333333;    // Variancia calculada com base na doc oficial do Arduino para o Gyro
float varAcc = 0.0000000166;    // Variancia calculada com base na doc oficial do Arduino para o Acc
float varMag = 0.0000653333;    // Variancia calculada com base na doc oficial do Arduino para o Mag

File arquivo;                   // Variavel arquivo
String nomeArquivo = "";        // Variavel do nome do arquivo
String blocoRegistro = "";      // Armazena 10 linhas antes de gravar
int count = 0;                  // Conta quantas amostras temos no buffer
```


FUNÇÃO DE KALMAN E ESCOLHA DO ARQUIVO

```
// 2. FUNÇÕES CRIADAS -----  
void kalman(float *kalmanEstimado, float *kalmanIncerteza, float taxa, float medida, float tempoDelta, float varAccMag){  
    //I. Predicao do estado: estimativa de onde o angulo estaria se so a entrada fosse considerada  
    *kalmanEstimado = *kalmanEstimado + tempoDelta * taxa;  
  
    //II. Predicao da incerteza: a incerteza cresce proporcionalmente ao tempo e ao ruido assumido do processo  
    *kalmanIncerteza = *kalmanIncerteza + tempoDelta * tempoDelta * varGyro;  
  
    //III. Calculo do ganho de Kalman: o ganho pondera quanto confiar na previsão vs na medida  
    float kalmanGain = *kalmanIncerteza / (*kalmanIncerteza + varAccMag); // varAccMag: variancia calculada de acordo com doc oficial  
  
    //IV. Correcao da estimativa e da incerteza  
    *kalmanEstimado = *kalmanEstimado + kalmanGain * (medida - *kalmanEstimado);  
    *kalmanIncerteza = (1 - kalmanGain) * (*kalmanIncerteza);  
}  
  
String NomeArquivo(){  
    int i=1;  
    String nome = "COLETA1.csv";  
  
    while(SD.exists(nome)){  
        i++;  
        nome = "COLETA" + String(i) + ".csv";  
    }  
  
    return nome;  
}
```


AQUISIÇÃO DOS ÂNGULOS DE ROTAÇÃO

```
// 4. LEITURAS -----  
void loop() {  
    // 4.1. Leitura do tempo ====  
    tempoAnterior = tempoAtual;  
    tempoAtual = millis();  
    tempoDelta = (tempoAtual - tempoAnterior) / 1000.0;  
  
    // 4.2. Leitura e ajuste do acelerometro ====  
    if(IMU.accelerationAvailable()){  
        IMU.readAcceleration(AccX, AccY, AccZ);  
  
        // Calculo dos angulos (roll e pitch) em graus  
        AnguloAccX = atan2(AccY, sqrt(AccX * AccX + AccZ * AccZ)) * 180.0 / PI;  
        AnguloAccY = atan2(-AccX, sqrt(AccY * AccY + AccZ * AccZ)) * 180.0 / PI;  
    }  
  
    // 4.3. Leitura e ajuste do giroscopio ====  
    if(IMU.gyroscopeAvailable()){  
        IMU.readGyroscope(GyroX, GyroY, GyroZ);  
  
        // Sensor retorna em rad/s: converter para graus/s  
        GyroX *= 180.0 / PI; GyroY *= 180.0 / PI; GyroZ *= 180.0 / PI;  
    }  
  
    // 4.4. Leitura do magnetômetro (Yaw absoluto) ====  
    if(IMU.magneticFieldAvailable()){  
        IMU.readMagneticField(MagX, MagY, MagZ);  
  
        // Calculo do angulo de Yaw absoluto (com base na Terra) em graus  
        AnguloMagZ = atan2(MagY, MagX) * 180.0 / PI;  
  
        // Correcao de angulos negativos para faixa 0°-360°  
        if (AnguloMagZ < 0) AnguloMagZ += 360.0;  
    }  
}
```

APLICAÇÃO DO KALMAN E GRAVAÇÃO FINAL

```
// 4.5. Aplicacao do Filtro de Kalman ====
kalman(&kalmanEstadoX, &kalmanIncertezaX, GyroX, AnguloAccX, tempoDelta, varAcc); // Roll
kalman(&kalmanEstadoY, &kalmanIncertezaY, GyroY, AnguloAccY, tempoDelta, varAcc); // Pitch
kalman(&kalmanEstadoZ, &kalmanIncertezaZ, GyroZ, AnguloMagZ, tempoDelta, varMag); // Yaw (corrigido com magnetometro)

// 4.6. Resultado final e impressao ====
rotX = kalmanEstadoX;
rotY = kalmanEstadoY;
rotZ = kalmanEstadoZ;

// Adiciona a linha ao bloco de registro
tempoSegundos = (tempoAtual - tempoInicial) / 1000; // -9000 pois eh o tempo medio para o Arduino comecar a gravar
blocoRegistro += String(rotX) + "," + String(rotY) + "," + String(rotZ) + "," + String(tempoSegundos, 2) + "\n";
count++;

// A gravacao do bloco no arquivo eh feita a cada 10 iteracoes (aprox 1s)
if(count == 10){
    arquivo = SD.open(nomeArquivo, FILE_WRITE);

    //Se o arquivo abriu, escreve o bloco de 1s de registro
    if(arquivo){
        arquivo.print(blocoRegistro);
        arquivo.close();
    }
    else while(1);

    // Limpa/reinicia as variaveis
    blocoRegistro = "";
    count = 0;
}

delay(100); // Pequeno atraso para estabilizar a taxa (~50 Hz)
```

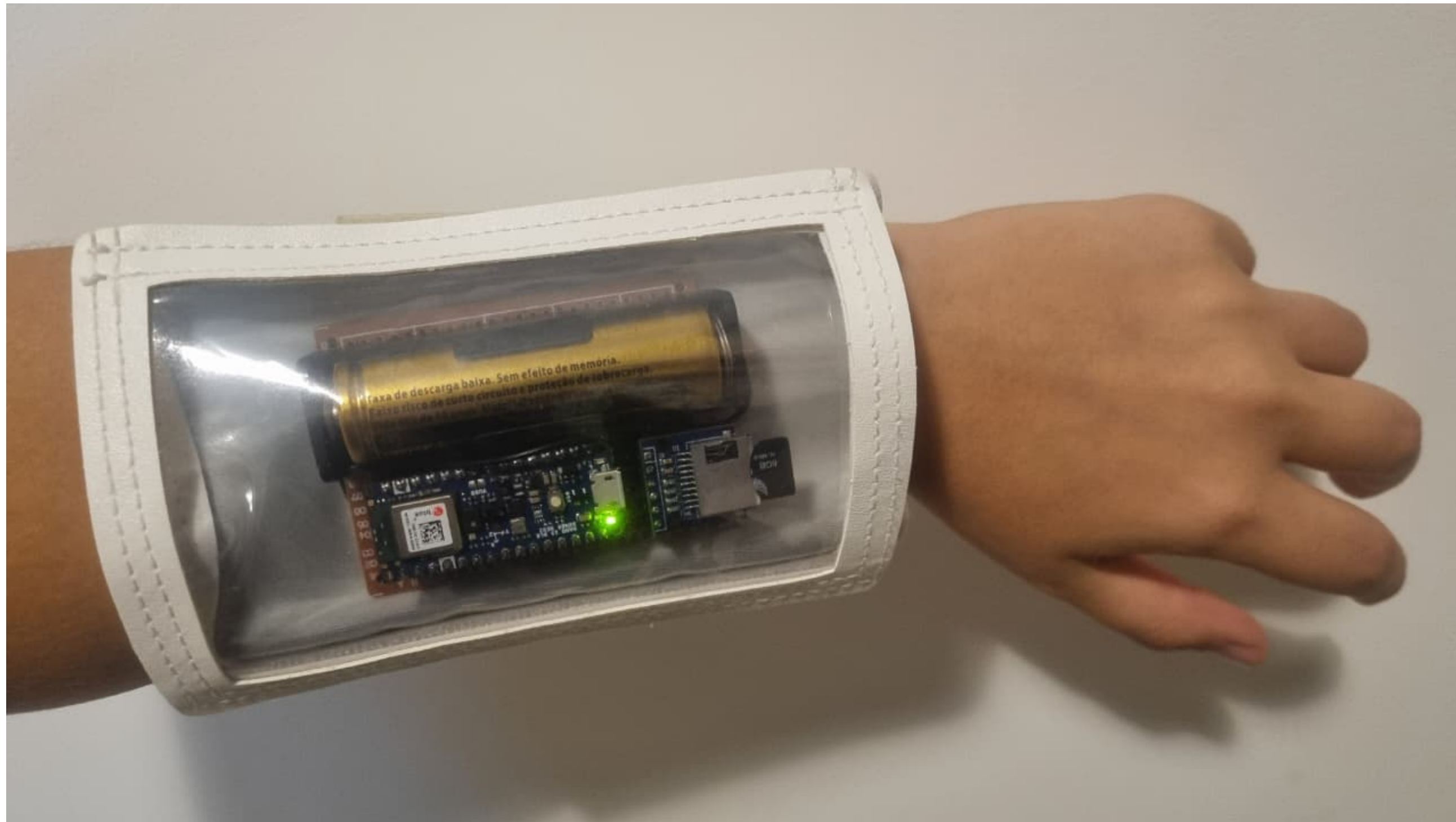
DURAÇÃO DA BATERIA

Para a análise teórica, adotou-se uma estimativa de consumo de corrente pessimista para os circuitos, da ordem de **100 mA**. Considerando a fonte de alimentação selecionada: baterias do tipo 18650 com capacidade nominal de **9800 mAh**, o cálculo de autonomia previa uma duração de aproximadamente **98 horas**.

Mesmo ao considerar capacidades menores, como **7500mAh**, e um consumo de corrente significativamente maior, como **250mA** (explicado através da energia dissipada através dos reguladores de tensão, por exemplo), a bateria ainda duraria **mais de um dia**.

- **Testes de validação:** três ensaios empíricos nos quais os protótipos, executando o *firmware* final, foram deixados em operação autônoma até a exaustão completa de diferentes unidades de bateria. O tempo de cada processo foi cronometrado e analisado.

SOLDA E SISTEMA FINAL AUTÔNOMO NO BRACELETE



Fonte: elaborada pela autora

PROTOCOLO DE TESTES MECÂNICOS

Comparação quantitativa robusta estímulo de entrada idêntico, controlável e repetível.

Plataforma de testes (*rig*) atuada por um servo motor MG90S. Este sistema gera **oscilações senoidais programadas**, simulando as características de frequência (tipicamente de 3 a 5 Hz na DP) e amplitude dos tremores. O sinal de comando enviado ao atuador estabelece a **ground truth** do experimento, permitindo uma aferição direta do erro de medição de cada protótipo.

A: ensaio de varredura de frequência. Quantifica a precisão da detecção no domínio da frequência (FFT);

B: ensaio de fidelidade de sinal, para analisar a acurácia da forma de onda no domínio do tempo (RMSE);

C: teste estático de longa duração, para avaliar o ruído intrínseco e a deriva.

ENSAIO A: ANÁLISE DE PRECISÃO EM FREQUÊNCIA

Objetivo: quantificar a precisão de cada sistema em detectar a frequência fundamental de um movimento periódico.

Rig programado para executar um movimento senoidal com amplitude angular constante (20°), enquanto a frequência do ground truth foi sistematicamente variada.

O Teorema de Nyquist-Shannon (SHANNON, 1949) limita a frequência de medição a 5Hz para evitar *aliasing* (subamostragem).

O protocolo executado, com duração de 60 segundos para cada ensaio, consistiu na varredura das seguintes frequências de ground truth: 2.0 Hz, 3.0 Hz, 4.0 Hz e 4.5 Hz.

ENSAIO B: ANÁLISE DE FIDELIDADE DA FORMA DE ONDA

Objetivo: avaliar a fidelidade dos protótipos no domínio do tempo.

Para isolar a resposta à amplitude, a frequência do ground truth foi mantida constante em 2.0 Hz, um valor cuja precisão foi validada no ensaio anterior e que se encontra dentro da faixa de interesse. O protocolo consistiu na execução de três testes de 60 segundos cada, variando-se sistematicamente a amplitude angular programada no servo motor, em 10, 20 e 30 graus.

A análise subsequente foca na comparação direta entre o sinal medido por cada protótipo e um sinal senoidal de referência (gerado digitalmente). Esta comparação utiliza a Normalização Z-score, para equiparar as escalas e focar puramente na forma, e o Erro Quadrático Médio (RMSE) como métrica quantitativa da fidelidade do sinal.

ENSAIO C: ANÁLISE DE ESTABILIDADE ESTÁTICA

Objetivo: avaliar o ruído intrínseco de cada sensor e a estabilidade da estimativa de orientação ao longo do tempo, fenômeno conhecido como deriva (drift).

Os protótipos foram acionados por um período de 30 minutos na plataforma de testes em repouso absoluto sobre uma superfície nivelada. Nesta condição, o sinal de ground truth para todos os eixos angulares é teoricamente zero.

Este teste de longa duração é **fundamental para validar a eficácia do Filtro de Kalman** implementado em cada firmware, permitindo uma comparação direta da acumulação de erro no eixo de guinada (Yaw). Este eixo, por não ser corrigido pelo vetor gravitacional do acelerômetro, depende da estabilidade do giroscópio e, no caso do Nano, da referência absoluta do magnetômetro para sua estabilização.

ANÁLISE COMPARATIVA DOS SINAIS INERCIAIS

Etapa conduzida em ambiente Python (3.12), com o suporte de bibliotecas especializadas para manipulação de dados e computação científica, como *Pandas*, *NumPy*, *SciPy* e *Matplotlib*.

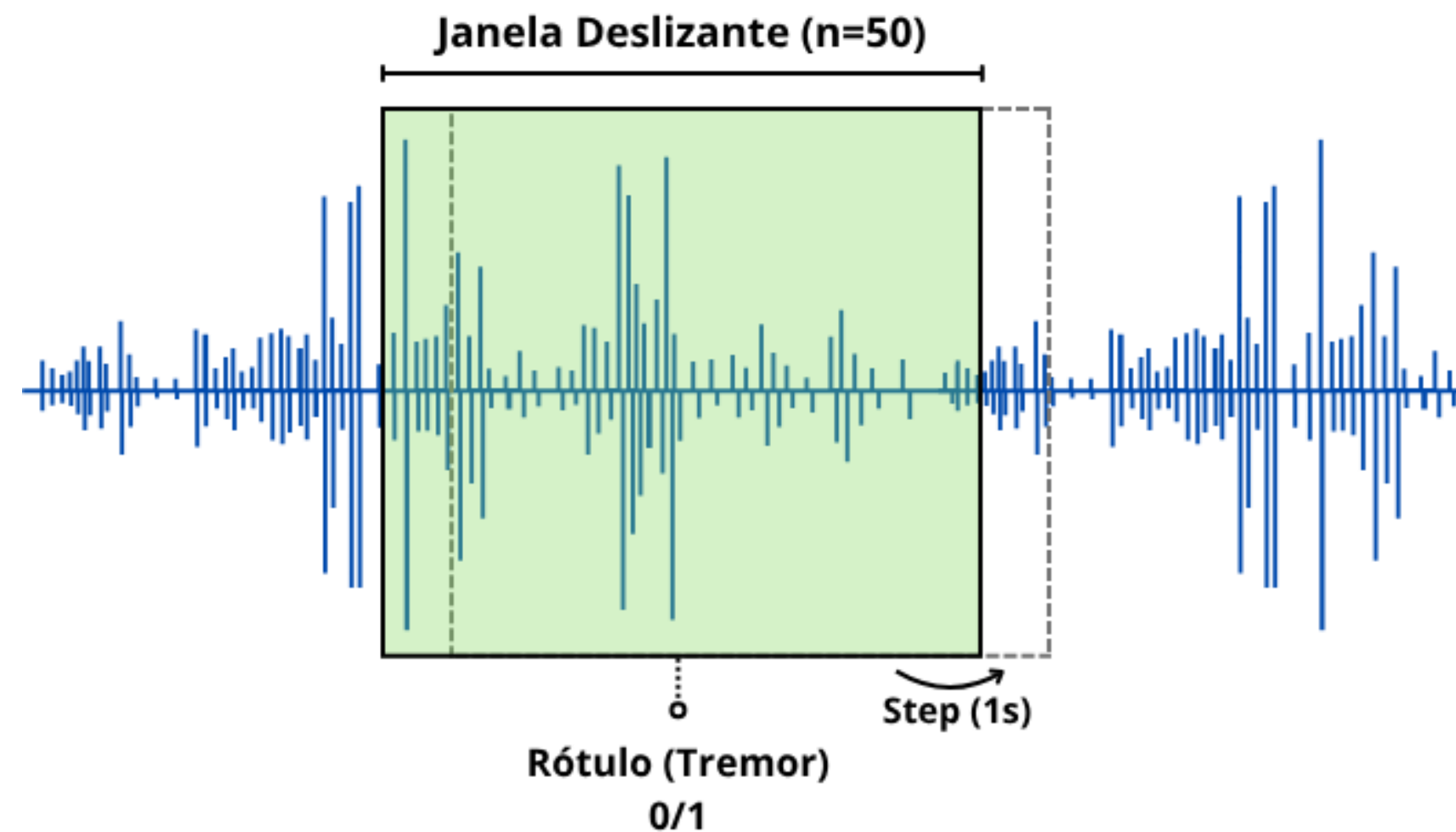
Ensaio A: análise espectral através da Transformada Rápida de Fourier.

Ensaio B: análise temporal pelo Erro Quadrático Médio (RMSE). O RMSE é uma medida padrão da diferença entre valores preditos por um modelo (ou medidos por um sensor) e os valores reais observados. Os sinais (medido e *ground truth*), foram submetidos à Normalização Z-Score, que ajustou os dados para garantir que a comparação de RMSE fosse justa e focada puramente na forma da onda, em vez de sua amplitude absoluta.

Ensaio C: quantificador de ruído intrínseco por desvio padrão, e quantificador de deriva gráfico.

PRÉ-PROCESSAMENTO DOS DADOS

Divisão estratégica dos dados, engenharia de features, normalização e geração de sequências (janelamento).



Fonte: elaborada pela autora

APLICAÇÃO DA ARQUITETURA LSTM

A arquitetura do modelo é composta pelas seguintes camadas:

1. **Camada de Entrada:** Define o formato de entrada esperado.
2. **Camada LSTM:** A primeira camada LSTM (com 64 unidades) processa a sequência de entrada.
3. **Camada de Regularização:** Uma camada Dropout é inserida para reduzir o sobreajuste, desativando aleatoriamente uma fração dos neurônios durante o treinamento.
4. **Segunda Camada LSTM:** Uma segunda camada LSTM recebe a sequência da camada anterior. Ela retorna apenas o último estado oculto, que resume toda a sequência de entrada.
5. **Camada Densa:** Uma camada totalmente conectada atua como um classificador, aprendendo combinações não-lineares do vetor de características gerado pela LSTM.
6. **Camada de Saída:** A camada final comprime a saída em um valor entre 0 e 1, que é interpretado como a probabilidade da sequência de entrada pertencer à classe Tremor.

METODOLOGIA DE TREINAMENTO E AVALIAÇÃO

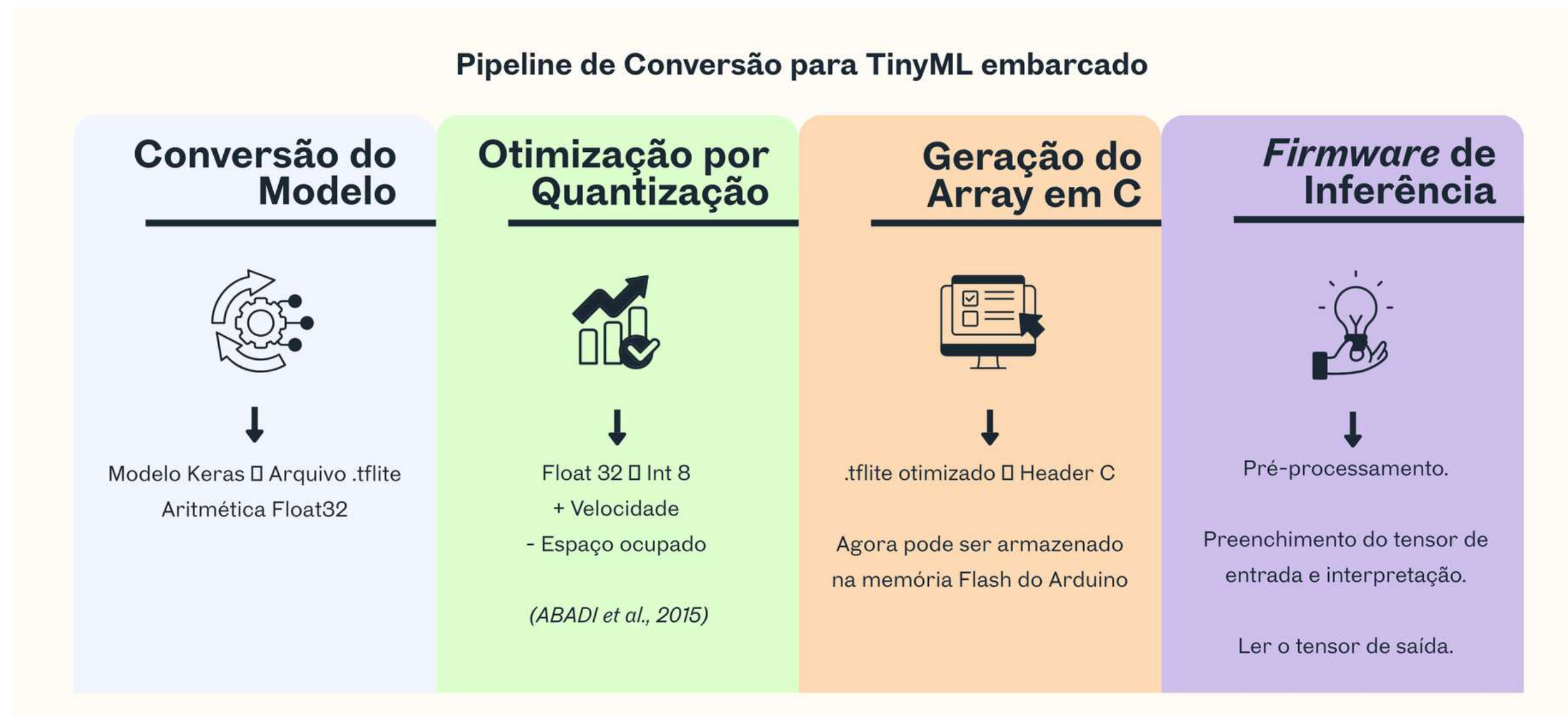
- Proporção treinamento-teste de **80/20**. O desempenho neste conjunto de validação é o indicador principal para diagnosticar o sobreajuste (overfitting);
- **Avaliação Final:**
 1. Acurácia: A proporção de classificações corretas (Tremor e Não-Tremor).
 2. Matriz de Confusão: Uma tabela que detalha os Verdadeiros Positivos, Falsos Positivos, Verdadeiros Negativos e Falsos Negativos.
 3. Precisão (*Precision*): Das janelas classificadas como tremor, quantas eram realmente tremor.
 4. Revocação (*Recall*): Das janelas que eram realmente Tremor, quantas o modelo conseguiu identificar.
 5. *F1-Score*: A média harmônica entre Precisão e Revocação.

METODOLOGIA DE TREINAMENTO E AVALIAÇÃO

- Proporção treinamento-teste de **80/20**. O desempenho neste conjunto de validação é o indicador principal para diagnosticar o sobreajuste (overfitting);
- **Avaliação Final:**
 1. Acurácia: A proporção de classificações corretas (Tremor e Não-Tremor).
 2. Matriz de Confusão: Uma tabela que detalha os Verdadeiros Positivos, Falsos Positivos, Verdadeiros Negativos e Falsos Negativos.
 3. Precisão (*Precision*): Das janelas classificadas como tremor, quantas eram realmente tremor.
 4. Revocação (*Recall*): Das janelas que eram realmente Tremor, quantas o modelo conseguiu identificar.
 5. *F1-Score*: A média harmônica entre Precisão e Revocação.

TRANSCRIÇÃO PARA TINYML

O processo de migração do modelo Keras (.h5) para um formato executável em microcontrolador seguiu um pipeline de conversão padrão do TensorFlow Lite (ABADI et al., 2015).



Fonte: elaborada pela autora.

RESULTADOS



SÍNTESE

- Testes de duração energética indicam infidelidade na capacidade indicada de baterias e superioridade proporcional do Arduino NANO. Assim, recomenda-se futuros estudos que visem fontes de alimentação mais robustas e duráveis.
- Testes motores indicam superioridade na captura dos sensores integrados ao Arduino NANO. O uso do magnetômetro, apesar de deixar as coletas significativamente mais ruidosas (principalmente em *Yaw*), elimina quase que completamente a deriva nesse eixo.
- A escalabilidade do Arduino NANO apresenta grande potencial de expansão e pesquisa, contribuindo para seu uso clínico.

COMPARATIVO DE HARDWARE E CUSTO

Protótipo UNO: Dimensões de 8cm x 12cm x 2cm e peso total de 123,9g.

Protótipo NANO: Dimensões de 5cm x 7cm x 2cm e peso total de 54,7g.

Solução Pedagógica/Verificação de conceitos
x
Solução Clínica

Protótipo pUNO_v2	
Componente	Preço (R\$)
Arduino UNO R3	37,00
MPU6050 (Giroscópio/Acelerômetro)	35,90
Módulo Micro SD 5V	18,90
Suporte 2x 18650 (em série)	15,90
Baterias 18650 9800 mAh (2x)	29,80
Cartão Micro SD 8GB	29,70
Placa Perfurada 8x12cm	8,30
CUSTO TOTAL (pUNO_v2)	175,50
Protótipo pNANO_v2	
Componente	Preço (R\$)
Arduino Nano 33 BLE Sense Rev2	415,00
Módulo Micro SD Mini 3.3V	19,00
Suporte 18650 (uma única)	14,90
Bateria 18650 9800 mAh (1x)	14,90
Cartão Micro SD 8GB	29,70
Placa Perfurada 5x7cm	4,50
CUSTO TOTAL (pNANO_v2)	498,00

COMPARATIVO DE CONSUMO ENERGÉTICO

A análise teórica do orçamento de potência projetou uma autonomia de bateria superior a 98 horas, com base na capacidade nominal de 9800 mAh das baterias 18650.

Nº Teste	Duração Nano	Duração UNO
1	4 horas e 31 minutos	7 horas e 3 minutos
2	4 horas e 23 minutos	6 horas e 45 minutos
3	5 horas e 36 minutos	5 horas e 57 minutos

A autonomia real representa menos de 10% do valor teórico.

Os dados reforçam a tese de ineficiência do UNO: mesmo com o dobro da energia disponível, o protótipo não alcançou o dobro da autonomia. Isso indica que o sistema UNO drena sua fonte de energia a uma taxa consideravelmente mais elevada, confirmando sua menor eficiência energética.

COMPARATIVO DE CAPTURA MOTORA

Tabela 1 – Resultados do Ensaio A: Análise de Precisão em Frequência (FFT)

Ground Truth (Hz)	<i>pUNO_v2</i>		<i>pNANO_v2</i>	
	Freq. Medida (Hz)	Erro (%)	Freq. Medida (Hz)	Erro (%)
2.0	2.01	0.56	1.99	-0.32
3.0	2.18	-27.40	2.99	-0.32
4.0	0.02	-99.58	3.99	-0.32
4.5	3.67	-18.54	0.22	-95.20

Tabela 2 – Resultados do Ensaio B: Análise de Fidelidade da Forma de Onda (RMSE)

Amplitude (Ground Truth)	RMSE <i>pUNO_v2</i>	RMSE <i>pNANO_v2</i>
10 graus	1.6520	2.1173
20 graus	1.3291	1.5577
30 graus	1.1732	0.7373

Tabela 3 – Resultados do Ensaio C

Protótipo	Roll (σ)	Pitch (σ)	Yaw (σ)	Protótipo	Deriva após 30 minutos
<i>pUNO_v2</i>	0.1638	0.2714	0.1037	<i>pUNO_v2</i>	1.56
<i>pNANO_v2</i>	0.0960	0.0642	1.7586	<i>pNANO_v2</i>	-0.18

VIABILIDADE DO *TINYML* E ESCALABILIDADE

- Para a análise offline, embora **ambos sejam funcionalmente viáveis**, apenas o NANO fornece a qualidade de dados necessária para que um modelo LSTM produza resultados clinicamente relevantes e confiáveis.
- A implementação do TinyML permite que o wearable realize a classificação dos tremores em tempo real, diretamente no dispositivo. Isso oferece vantagens substanciais em termos de **privacidade** (dados brutos não saem do dispositivo), **consumo de energia** (a transmissão de dados é minimizada) e **resposta imediata**.
- Além disso, NANO é absolutamente mais escalável.

A implementação de um modelo LSTM para classificação em tempo real via TinyML é não apenas viável, mas uma das principais vantagens estratégicas do NANO, validando-o como uma plataforma de hardware adequada para o desenvolvimento de um sistema assistivo inteligente e autônomo.

CONSIDERAÇÕES FINAIS



- O presente trabalho concluiu com êxito o desenvolvimento e a validação comparativa multifacetada de dois protótipos de wearables para o monitoramento de tremores na Doença de Parkinson, endereçando os objetivos propostos.
- O protótipo UNO se saiu surpreendentemente bem em testes, mas para um ambiente clínico, onde a precisão, conforto e privacidade são fatores cruciais, a superioridade tecnológica consolida o protótipo NANO como melhor sistema.
- Identificam-se limitações metodológicas, como a taxa de amostragem (10Hz) e a duração das baterias. Além disso, futuros trabalhos poderiam incluir a validação clínica do(s) dispositivo(s), com pacientes.

REFERÊNCIAS



ABADI, M. et al. TensorFlow: large-scale machine learning on heterogeneous systems. 2015. Software available from tensorflow.org. Disponível em: <https://www.tensorflow.org/>. Acesso em: 08 out. 2025.

AMERINI, I. et al. Robust smartphone fingerprint by mixing device sensors features for mobile strong authentication. Electronic Imaging, v. 28, n. 8, p. 1–8, 14 fev. 2016.

Arduino. One board to rule them all: History of the Arduino UNO. 2021. Acesso em: 30 mar. 2025. Disponível em: <<https://blog.arduino.cc/> 2021/12/09/one-board-to-rule-them-all-history-of-the-arduino-uno/?queryID=582dd1d4ea530a77d7988f12f7a9311d>.

ARMSTRONG, M. J.; OKUN, M. S. Diagnosis and treatment of parkinson disease: a review. JAMA, v. 323, n. 6, p. 548–560, fev. 2020.

Cala Health. Cala Trio Therapy. 2024. Acessado em: 06 ago. 2025. Disponível em: <https://calahealth.com/support/help-center/about-cala/cala-trio-therapy/>.

FEIGIN, V. L. et al. The global burden of neurological disorders: translating evidence into policy. The Lancet Neurology, v. 19, n. 3, p. 255–265, mar. 2020.

Great Lakes NeuroTechnologies. Kinesia 360. 2024. Acessado em: 06 ago. 2025. Disponível em: <https://www.glneurotech.com/products/kinesia-360/>.

Gyro Sensors | Technology | Epson. Disponível em: [<https://corporate.epson/en/technology/overview/microdevice/gyro-sensor.html>] (<https://corporate.epson/en/technology/overview/microdevice/gyro-sensor.html>). Acesso em: 12 nov. 2025.

HOCHREITER, S.; SCHMIDHUBER, J. Long short-term memory. *Neural Computation*, v. 9, n. 8, p. 1735–1780, dez. 1997.

KLEIN, M.; BATTAGELLO, S.; CARDOSO, A. Dopamine: functions, signaling, and association with neurological diseases. *Cellular and Molecular Neurobiology*, v. 39, p. 31–59, 2019.

LIN, C.; YEAP, T.; KIRINGA, I. Stacked Bidirectional LSTM for Predicting Emission of Nitrous Oxide. *Proceedings of the Canadian Conference on Artificial Intelligence*, 27 maio 2022.

MITCHELL, T. *Machine Learning*. London: Mcgraw-Hill Pub. Co. (Ise Editions), 1997.

ROVINI, E.; MAREMMANI, C.; CAVALLO, F. How wearable sensors can support parkinson's disease diagnosis and treatment: a systematic review. *Frontiers in Neuroscience*, v. 11, 2017.

SHANNON, C. E. Communication in the presence of noise. *Proceedings of the IRE, IEEE*, v. 37, n. 1, p. 10–21, jan. 1949. Disponível em: <https://doi.org/10.1109/jrproc.1949.232969>.

SONER SONMEZOGLU et al. Dual-resonator MEMS Lorentz force magnetometer based on differential frequency modulation. p. 160–163, 1 mar. 2017.

SUN, N. et al. Single-nucleus multiregion transcriptomic analysis of brain vasculature in Alzheimer's disease. *Nature neuroscience*, v. 26, n. 6, p. 970–982, jun. 2023.

WALKER, R. et al. Cue Band, Wearables for Parkinson's. 2024. Acessado em: 02 ago. 2025. Disponível em: <https://cue.band/>.

ZAMBOTTI, M. D. et al. State of the science and recommendations for using wearable technology in sleep and circadian research. *Sleep*, v. 47, n. 4, dez. 2023. ISSN 0161-8105.